

SPAM SMS DETECTION USING NLP AND MACHINE LEARNING

Student Name: Siri Vennela Marati

Student Id: 23080951

Course: MSc Data Science with Sandwich Placement

GitHub Link: <https://github.com/maratisiri/Machine-learning-.git>

Table of Contents:

- Introduction
- Objectives
- Data Summary
- Techniques
 - Data preprocessing
 - (TF-IDF Vectorization) Feature engineering
 - Model training
 - Evaluation
- Advanced Visualization
- Outcome
- Conclusion
- References

INTRODUCTION:

Spam SMS is a crucial part of electronic communication. Many users collect unwanted messages that are either spam or promotions. The increase in spam has not only made inboxes cluttered but also posed a remarkable security risk including malware distribution. To clash this, various methods have been developed with ML and NLP to become evident as effective tools for automating spam detection using NLP (natural language processing) and machine learning. We evaluate messages, train a classifier and examine performance using real-world datasets.

OBJECTIVES:

Using NLP develop spam detection.

Use of Machine Learning Classifiers to categorize sms as spam or ham.

Apply text preprocessing to handle missing values, and duplicates and to clean raw data into cleaned data in a structured format.

Analyse Model performance using accuracy and visualization for a better understanding of analysis.

DATA SUMMARY:

We use the SMS Spam Collection Dataset from Kaggle.

Dataset Link: [SMS Spam Collection Dataset](#)

ATTRIBUTES OF DATASET:

Removed irrelevant columns from the data and named the column name as Label and Messages, to identify the messages we used mapping, if the label is spam (1) or ham(0). In the message column it returns the original text content.

```
# removing unnecessary columns
df = df.iloc[:, :2]
df.columns = ['label', 'message']

# Convert labels to binary values (ham = 0, spam = 1)
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Display first few rows
print(df.head())
```

	label	message
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

TECHNIQUES:

➤ DATA PREPROCESSING:

By using the function removed the punctuation, and stop words, converted the text into lowercase and cleaned the text.

TEXT CLEANING:

```
[9]: # Define def function to clean the text at once
def clean_all_text(text):
    text = text.lower() # LOWERCASE
    text = re.sub(f"[{string.punctuation}]", "", text) # Remove punctuation
    tokens = word_tokenize(text) # Tokenize text
    tokens = [word for word in tokens if word not in stopwords.words('english')] # Remove stopwords
    return " ".join(tokens)

# Applying text cleaning function to the message column
df['clean_message'] = df['message'].apply(clean_text)

# print first few cleaned messages
print(df[['message', 'clean_message']].head())
```

	message	clean_message
0	Go until jurong point, crazy.. Available only ...	go jurong point crazy available bugis n great ...
1	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	Free entry in 2 a wkly comp to win FA Cup fina...	free entry 2 wkly comp win fa cup final tkts 2...
3	U dun say so early hor... U c already then say...	u dun say early hor u c already say
4	Nah I don't think he goes to usf, he lives aro...	nah dont think goes usf lives around though

- **FEATURE ENGINEERING:** This is used to develop the performance of machine learning models by changing raw data into cleaned data to get exact accuracy. Following spam sms often has different features, such as unusual sender details, specific keywords, and feature engineering to assist extract this sample for good classification. Techniques like text vectorization (for example word embeddings, TF-IDF) are used to extract email meta information and recognise spam-related words adding to the model's capacity to distinguish between spam and authorized messages. In the code, we convert text into numerical form using TF-IDF. Before feature engineering, we need to divide the data into training (80%) and testing sets (20%), dividing it into sets will make sure a model learns the design (training) and assess unseen data (testing). This stops overfitting, makes certain unbiased performances, and makes better generalizations. In spam sms detection, it helps out model identify spam based on designs instead of memorization.

splitting the dataset into training and testing, (80%training, 20% testing)

```
: # splitting the train and testing
X_train, X_test, y_train, y_test = train_test_split(df['clean_message'], df['label'], test_size=0.2, random_state=42)
```

Convert Text to Numerical Data Using TF-IDF

```
: # Converting text data into TF-IDF features
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

MODEL TRAINING:

Model training in spam sms detection is important to instruct the machine-learning algorithm on how to differentiate between spam and authorized messages. By providing the model-labelled data (spam and ham emails), it grasps patterns, such as special characters, word constancy and sender information. This training allows the model to make exact predictions on new, without training the model we cannot identify spam effectively. In this code, we split the dataset into train and testing and we train a naïve Bayes classifier.

Train a Machine Learning Model

```
# use Train Naïve Bayes(multinomial) classifier
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)
```

▼ MultinomialNB
MultinomialNB()

MODEL EVALUATION:

Model evaluation in spam sms detection is important to evaluate how well the trained model differentiates between spam and authorized messages. This assists in measuring exactly, recalling, and precision and F1-score, making sure the model performs successfully on real-world data. Without estimation, there is no other way to identify if the model is overfitting. Proper estimation helps fine-tune the model, enhance spam detection exactly and reduce positive errors (authorized emails are noticeable as spam) and incorrect (spam emails are noticeable as authorized). In the code, we will check the model exactly and create a classification report.

Evaluate the Model

```
4]: accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Print classification report
print('\nClassification Report:\n', classification_report(y_test, y_pred))

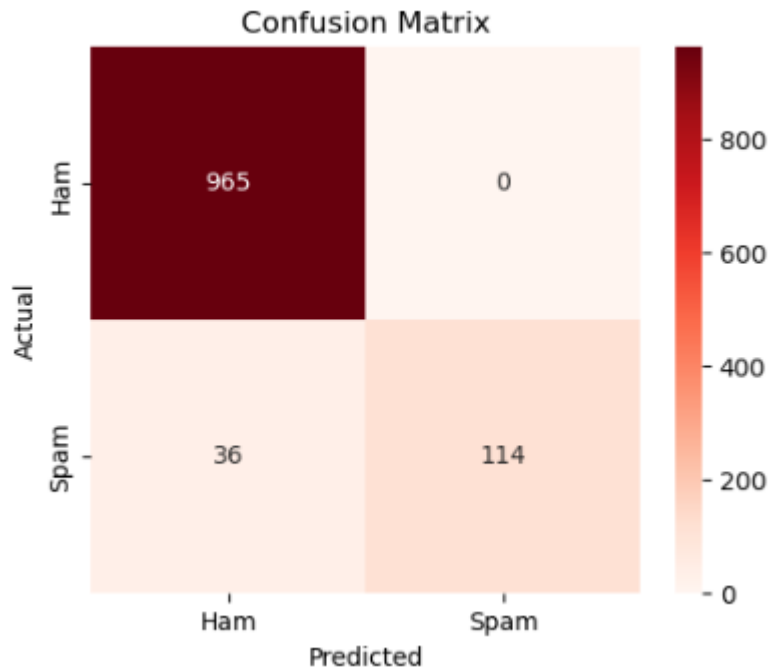
# Display confusion matrix as a heatmap
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Reds', xticklabels=['Ham', 'Spam'], yticklabels=['Ham', 'Spam'],
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Accuracy: 96.77%

Classification Report:				
	precision	recall	f1-score	support
0	0.96	1.00	0.98	965
1	1.00	0.76	0.86	150
accuracy			0.97	1115
macro avg	0.98	0.88	0.92	1115
weighted avg	0.97	0.97	0.97	1115

ADVANCED VISUALIZATION:

➤ CONFUSION MATRIX:



The confusion matrix gives insights into how well your model (Naïve Bayes classifier) performs on spam detection, this confusion matrix contains four values,

1) True Positives 2) True Negatives 3) False Positives 4) False Negative

In True positives, the value 114 indicates which means spam messages were correctly classified as spam.

In True Negatives, authorized messages are correctly classified as non-spam(ham) with a value of 965 indicating the highest value.

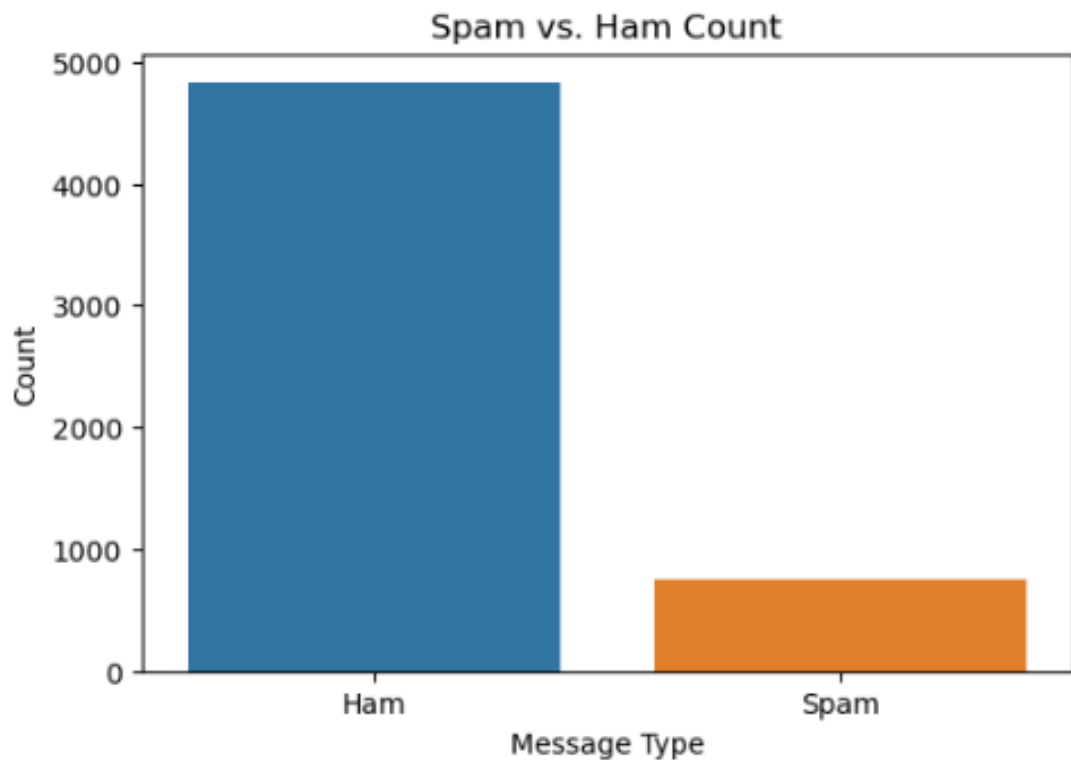
In False Positive, no authorized messages were wrongly classified as spam and it indicates the value of 0.

In False Negative it shows a 36 value which means relatively high, spam messages wrongly classified as ham.

➤ **COUNT PLOT:**

```
n [28]: #COUNT PLOT
import seaborn as sns
import matplotlib.pyplot as plt

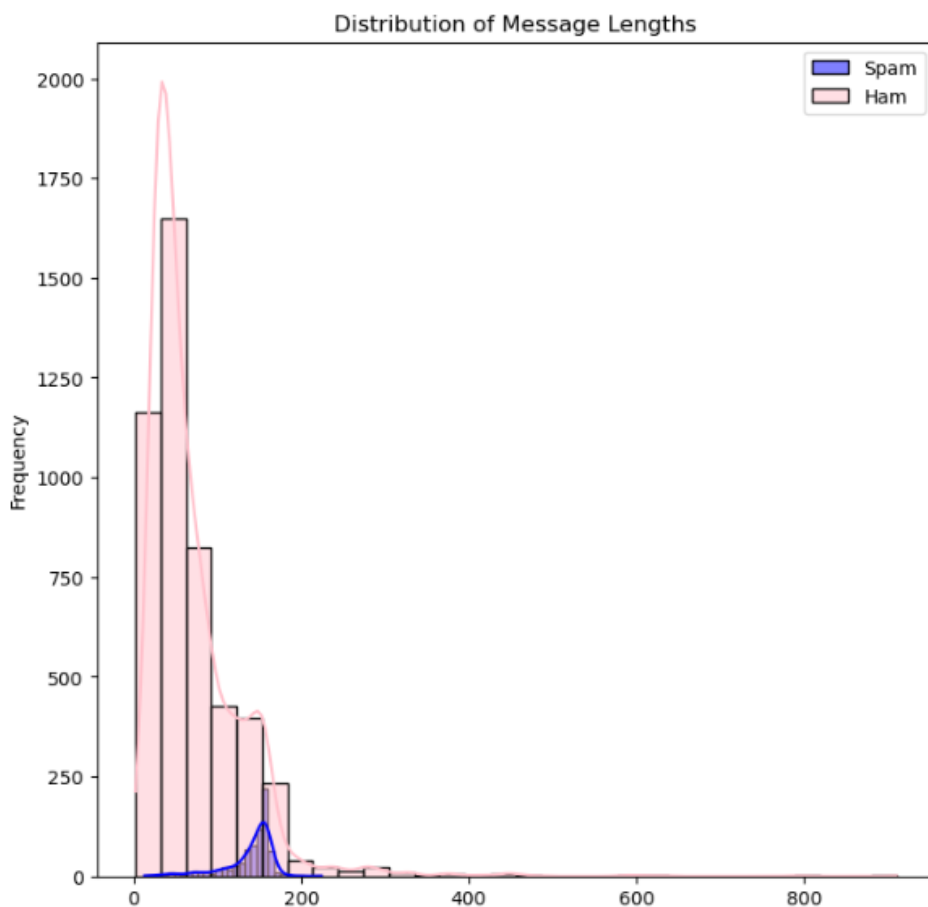
plt.figure(figsize=(6,4))
sns.countplot(x=df['label'])
plt.title("Spam vs. Ham Count")
plt.xlabel("Message Type")
plt.ylabel("Count")
plt.xticks(ticks=[0,1], labels=["Ham", "Spam"])
plt.show()
```



We used a count plot to check the balance of the dataset like several spams vs ham. If there are outstandingly more ham messages than spam datasets, the dataset is unstable. If a dataset is unstable this could influence model performance. We can observe there is a leading bias towards ham messages compared with spam.

➤ HISTOGRAM FOR TEXT LENGTH DISTRIBUTION:

```
[36]: #text length distribution
plt.figure(figsize=(8,8))
sns.histplot(df[df['label'] == 1]['message'].apply(len), bins=30, kde=True, color='blue', label="Spam")
sns.histplot(df[df['label'] == 0]['message'].apply(len), bins=30, kde=True, color='pink', label="Ham")
plt.legend()
plt.title("Distribution of Message Lengths")
plt.xlabel("Message Length (Characters)")
plt.ylabel("Frequency")
plt.show()
```



We used a histogram plot to check spam messages are liable to have more words than normal messages. In the plot, we can observe that spam messages are liable to be longer than ham messages which means it contains more promotions, and many ham messages are shorter, specifying simple conversations.

➤ WORD CONSTANCY VISUALIZATION:

We used the word cloud to recognise the most common words in spam and ham messages. Spam messages hold words like 'fire', 'win', 'offer', 'prize', 'cash', 'mobile', 'call', 'please' and 'click', These words identify as promotional content. Ham messages recognise the most natural words 'home', 'come',

RESULTS:

[illegible]

Model Accuracy: By performing model Evaluation we got model accuracy as 97.5%.

Spam Messages contain: 'fire', 'win', 'offer', 'prize', 'cash', 'mobile', 'call', 'please' and 'click'.

Ham Messages contain: 'home', 'come', 'time', 'got', 'need', 'today', 'going', 'ok', 'work', 'go', 'don't'.

The word cloud visualization helps us a lot to identify spam messages focusing on promotions.

CONCLUSION:

The spam sms detection system effectively identifies spam messages with high exactness using NLP techniques. By influencing TF-IDF Vectorization and Naïve Bayes Classification, the model successfully distinguishes ham and spam messages. The advanced visualization techniques also give in-depth insights into designs within spam messages, helping in feature selection and model development.

REFERENCES:

Kowsari, K., Meimandi, K. J., Heidarysafa, M., Mendu, S., Barnes, L. E., & Brown, D. E. (2019), Text classification techniques.

Rennie, J. D., Shih, L., Teevan, J., & Karger, D. R. (2003) focused on Naïve Bayes classifiers.